

Title: AI Powered Aircraft Health and Efficiency Dashboard

Group Members:

1. Areeba Khan (SE221032)
2. Ruhab A. Rahim (SE221029)

Overview:

Our project aims to create a **Digital Twin-lite** environment for aircraft fleet management. By merging three distinct "data silos"—Real-time flight movements (ADS-B), Environmental conditions (METAR), and Engine physics (NASA CMAPSS)—you are building a predictive engine that answers two critical questions for an airline:

1. **"How healthy is my fleet?"** (Predicting maintenance before it becomes a failure).
2. **"How green is my operation?"** (Optimizing fuel and CO2 based on real-world flight profiles).

Instead of just being a "black box" model, your project emphasizes **Interpretability** through a dashboard (Streamlit/Dash), allowing operators to see *why* a flight is high-risk or where fuel was wasted. It is a research-grade prototype that proves AI can transform open-source aviation data into actionable business intelligence.

Major Issues & Strategic Solutions

1. The "Data Silo" Synchronization Problem (Temporal Alignment)
 - **The Issue:** We are dealing with data at different "speeds." ADS-B data is high-frequency (seconds), METAR is hourly, and CMAPSS engine cycles are per-flight. Trying to join these on a single table often results in "data explosion" or massive gaps where weather doesn't match the flight's exact location at the right time.
 - **The Solution:** Implement a **Feature Aggregation Layer**. Instead of joining raw data, summarize it at the **"Flight Segment"** level. For every Flight ID, calculate: Average altitude, Max Speed, Predominant Weather Category, and Cumulative Engine Stress. This reduces our data volume and ensures all variables are "talking the same language" in your ML model.
2. The "Sensor Gap" (OpenSky vs. CMAPSS)
 - **The Issue:** NASA's CMAPSS data relies on internal engine sensors (Temperatures, Pressures, Fuel Flow). However, OpenSky (ADS-B) only gives you external telemetry (Altitude, Speed, Heading). We cannot "see" the engine's internal heat from an ADS-B ping.

- **The Solution: Use Proxy Variables & Load Profiling.** Since we lack internal sensors, we must calculate "Derived Load." Use the aircraft's Rate of Climb and Acceleration against its Altitude to estimate how hard the engine is working. Map these "Proxy Loads" to the CMAPSS degradation curves to estimate the RUL (Remaining Useful Life).
3. The "Synthetic Data Loop" (Model Circularity)
 - **The Issue:** Since we are generating synthetic maintenance logs using rules, there is a danger that your ML model will simply "learn the rules" we wrote, rather than learning actual patterns. If our rule is "If $RUL < 10$, then Maintenance = Yes," the ML model will achieve 100% accuracy, but it will be useless in the real world.
 - **The Solution: Introduce Probabilistic Noise and "Stochastic Failures."** When generating synthetic data, use a "Gaussian Noise" factor. Make it so that some engines fail "early" (random faults) and some last longer than expected. This forces the ML model to look for deeper correlations (like weather stress + high utilization) rather than just reading a single threshold.
 4. The "Fleet Normalization" Bias
 - **The Issue:** An Airbus A380 and a Boeing 737 have vastly different fuel burns and maintenance schedules. If we train one model on a mixed fleet, the "Efficiency Index" will always flag the larger aircraft as "Inefficient" simply because of its size.
 - **The Solution: Use Categorical Embedding or Normalization by Peer Group.** Our model must compare an aircraft's performance against the average for its specific type and route. Instead of "Fuel Burned," our target variable should be "**Deviation from Fleet Mean**" for that specific aircraft model.
 5. Dashboard Latency and "The Streamlit Bottleneck"
 - **The Issue:** Streamlit is fantastic, but it re-runs the entire script every time a user moves a slider. If our "Unified Dataset Builder" is processing 100,000 flight points in the background, our dashboard will lag, freeze, and provide a poor user experience.
 - **The Solution: Use Parquet Files and Caching.** Do not process raw CSVs in the dashboard. Pre-calculate your scores and save them in a compressed format like .parquet (which is much faster than CSV). Use the `@st.cache_data` decorator in Streamlit so that once a score is calculated, it stays in memory, making the user experience feel "snappy" and professional.